AODjustify – Rapport (Programmation Dynamique & Localité Mémoire)

Prénom Nom – Ensimag

3 novembre 2025

1 Avant-propos et principe

Lisez d'abord la documentation Doxygen (`docs/html/index.html` dans le zip, ou `make index` pour la régénérer). Ce rapport vise l'essentiel : architecture, points chauds et performances.

But. `AODjustify` (TP 4MMAOD6) justifie un texte selon une largeur M. Entrée via `mmap` (texte ISO-8859-1), sortie `.out` écrite séquentiellement.

Modules.
- **`parse.c`** : scan `mmap`, paragraphes, mots.
- **`justify.c`** : DP (hot spot) & reconstruction des lignes.
- **`check.c`** : contrôles d'usage/fichier (encodage, bornes, args).
- **`out.c`** : écriture sortie (lignes, sauts de paragraphe).
- **`main.c`** : orchestration & gestion des erreurs.

2 Modélisation (rappel très bref)

Les Question 1 et Question 2 du sujet sont détaillées dans theorie.pdf (même dossier). Rappel minimal pour le contexte :
- largeur d'une ligne $i..j$: $\text{width}(i,j) = \sum_{k=i}^{j} W[k] + (j-i)$ (avec $\mu = 1$) ;
- coût de ligne : $C(i,j) = 0$ si $j = n-1$, sinon $(M - \text{width}(i,j))^3$ si $\text{width}(i,j) \le M$;
- Bellman : $J[i] = \min_{j \ge i,\ \text{width} \le M} \big(C(i,j) + J[j+1]\big)$, $J[n] = 0$.

3 Architecture et choix de conception

Paragraphes géants ($> 10^6$ mots) $\Rightarrow$ architecture cruciale.

3.1 Structures de données

Par paragraphe (n mots) :
- `P` (`char**`) : pointeurs vers les mots (`mmap`).
- `W` (`uint16_t*`) : longueurs de mots.
- `J` (`uint64_t*`) : coûts $J[i]$ (variante 64 bits).
- `N` (`uint64_t*`) : décision j (variante 64 bits).

Note. La version initiale utilisait `uint32_t` pour `J`/`N`; les mesures de cache ci-après sont avec J/N 64 bits. Pour $\text{width}(i,j)$, on utilise une somme glissante (`sumLen += W[j]`) plutôt qu'un préfixe `S[]` (moins de lectures).

3.2 Itératif vs. récursif

Top-down mémoïsé (stack) abandonné. Bottom-up ($J[n] \to J[0]$) retenu : contrôle précis des accès mémoire.

3.3 Streaming par paragraphe

Approche globale (tout en RAM) abandonnée. Traitement par paragraphe : empreinte $O(n_{\mathsf{max}})$ (et $O(S)$ virtuel pour `mmap`).

3.4 Localité & points coûteux

Table 1: Tableau de localité ($n = 1{,}209{,}348$, J/N 64 bits)

Zone	Taille	Écrit	Lu	Profil d'accès
`W`	$\sim$ 2.3 Mo	parse	DP	séquentiel (favorable L1)
`J` & `N`	$\sim$ 18.4 Mo	DP	DP/reconstr.	séquentiel (J) / aléatoire (N)
`P`	$\sim$ 9.2 Mo	parse	reconstr.	lecture dispersée
I/O (mmap/out)	$\sim$ 14.4 Mo	sortie	entrée	séquentiel, coût incompressible

Points coûteux (hotspots) : (1) DP dans `justify.c` (écrit `J,N`; lit `W,J`); (2) reconstruction dans `out.c` (suit `N`, lit `P,W`, écrit la sortie) ; (3) I/O séquentiel (scan d'entrée & écriture sortie), coût fixe.

4 Performance théorique (Q5)

Notations. S : taille du fichier (octets) ; N : nombre total de mots ; n_p : taille (mots) du plus gros paragraphe ; W_{max} : nb max de mots sur une ligne de largeur M.

4.1 Complexité temps & espace

- Espace : $O(S + n_p)$ (pages `mmap` côté OS, plus tableaux du plus gros paragraphe).[1]
- Temps : parsing+reconstruction en $O(S+N)$; DP en $O(n_p \cdot W_{\mathsf{max}})$ (boucle j coupée dès width $> M$) $\Rightarrow$ linéaire en pratique.

4.2 Bornes de cache (incompressibles)

- Entrée (mmap) : lectures séquentielles $\approx \lceil S/64 \rceil$ lignes de cache (L1 & LL).
- Sortie : écritures séquentielles $\approx \lceil S_{\text{out}}/64 \rceil$ lignes (write-back).
- Tableaux : écritures uniques de `P` (8o), `W` (2o), `J` (8o), `N` (8o) $\Rightarrow$ lignes modifiées $\approx \lceil (8+2+8+8) \cdot n_p/64 \rceil$.
  Passer `J/N` en 64 bits double la part `J/N` dans ces écritures $\Rightarrow$ plus de `DLmw`.

4.3 Lecture guidée des compteurs (extrait)

- LL total : $ILmr + DLmr + DLmw =$ **607 298** ; dont $DLmr =$ **113 906** (lecture incompressible) et $DLmw =$ **491 769** (écritures obligatoires, accrues par J/N 64 bits).
- L1 total : $I1mr + D1mr + D1mw =$ **1 153 291** ; borne streaming $\sim$ **716 602** ; surcoût (accès non séquentiels `N`/`P`) $\sim$ **435 052**.

[1] Ici $O(S)$ concerne l'espace virtuel du `mmap`, pas une copie en RAM.

5 Analyse expérimentale (Q6)

5.1 Temps d'exécution

Table 2: Temps moyens (ms) sur 5 runs

	Petits Fichiers			Gros Fichiers	
Benchmark	(-O0)	(-O2)	Benchmark	(-O0)	(-O2)
Fichier 1 (court)	4.9	4.7	Fichier original	139.5	76.2
Fichier 2 (foo 1l)	3.7	3.8	Taille x 10	1385.6	N/A
Fichier 3 (foo 2l)	4.0	4.0	Taille / 2	10.4	10.6
Fichier 4 (foo2)	4.0	4.0	Taille / 3	9.4	9.3
Fichier 5 (longtemps)	5.0	4.9	1 paragraphe	137.4	76.1

5.2 Misses totales

Table 3: Total Cache Misses (L1+LL, Data+Instr.)

	Petits Fichiers			Gros Fichiers	
Benchmark	(-O0)	(-O2)	Benchmark	(-O0)	(-O2)
Fichier 1 (court)	226,665	224,158	Fichier original	1,459,581	1,193,973
Fichier 2 (foo 1l)	217,731	219,270	Taille x 10	10,296,046	N/A
Fichier 3 (foo 2l)	219,762	220,700	Taille / 2	339,933	351,065
Fichier 4 (foo2)	221,347	221,163	Taille / 3	308,238	325,753
Fichier 5 (longtemps)	228,822	227,624	1 paragraphe	898,532	867,688

5.3 Lecture

`-O2` profite surtout aux gros fichiers (programme memory-bound). Multi-paragraphe vs mono-paragraphe : temps proches mais plus de misses en multi-paragraphe (malloc/memset répétés, pollution L1/LL).